



**Rajshahi University of Engineering & Technology
(RUET)**

Department of Electrical & Computer Engineering

Lab Report - 2

Course Title: Discrete Mathematics & Numerical Methods

Course Code: ECE 2214

Experiment Name:

Implementation of root finding by False Position Method

Submitted by,

Md. Yousuf

Roll : 2310023

Submitted to,

Md. Faysal Ahamed

Assistant Professor

Dept of ECE, RUET

Contents

1	Theory	2
2	Required Tools	2
3	Equations to Solve	2
4	Algorithm: False Position Method	2
5	Code Implementation	3
5.1	Code for Equation 1	3
5.2	Code for Equation 2	4
6	Output and Graphs	5
6.1	Program Outputs (Command Window)	5
6.2	Graphical Representation	5
7	Discussion	7
8	Conclusion	7

List of Figures

1	Octave Terminal Output and Iteration Table for Equation 1 (False Position)	5
2	Octave Terminal Output and Iteration Table for Equation 2 (False Position)	5
3	Plot of Equation 1: $f(x) = x^3 - 2x^2 - 5$	6
4	Plot of Equation 2: $f(x) = e^x - 4\sin(x) - 3\ln(x)$	6

1 Theory

The False Position (Regula Falsi) Method is an iterative root-finding technique that improves on the bisection method by using a secant line to estimate the root.

The root is approximated as:

$$c = a - f(a) \cdot \frac{b - a}{f(b) - f(a)} \quad (1)$$

Then:

- If $f(a) \times f(c) < 0$, the root lies in $[a, c] \rightarrow$ set $b = c$.
- If $f(a) \times f(c) > 0$, the root lies in $[c, b] \rightarrow$ set $a = c$.

This process repeats until the interval is sufficiently small or the desired accuracy is achieved.

2 Required Tools

1. Mac M4 Air (Hardware)
2. GNU Octave (Software environment)
3. MS Office / LaTeX (For report generation)

3 Equations to Solve

To demonstrate the method, we solve two distinct non-linear equations:

- **Equation 1 (Polynomial):** $f(x) = x^3 - 2x^2 - 5 = 0$
- **Equation 2 (Logarithmic, Trigonometric, Exponential):** $f(x) = e^x - 4 \sin(x) - 3 \ln(x) = 0$

4 Algorithm: False Position Method

1. Start
2. Input: equation $y(x)$, initial guesses a and b .
3. Set tolerance $lim = 0.001$.
4. Compute $fa = y(a)$ and $fb = y(b)$.
5. If $fa \times fb > 0$, display "Wrong guess" and stop.
6. Else:

(a) Compute approximation $c = a - fa \cdot \frac{b-a}{fb-fa}$.

- (b) Evaluate $fc = y(c)$.
- (c) While $|fc| > \text{lim}$:
- Print a, b, c, fc .
 - If $fa \times fc < 0$, set $b = c$ and $fb = f(b)$, else set $a = c$ and $fa = f(a)$.
 - Update $c = a - fa \cdot \frac{b-a}{fb-fa}$ and $fc = y(c)$.
- (d) Print root = c .

7. Stop

5 Code Implementation

Two separate and independent Octave scripts were developed to find the roots of the given equations using the False Position method.

5.1 Code for Equation 1

```

1 % False Position Method for Equation 1: f(x) = x^3 - 2*x^2 - 5
2 f = @(x) x^3 - 2*x^2 - 5;
3 a = 2; b = 3; % Initial guesses
4 lim = 0.001; % Tolerance limit
5
6 fa = f(a);
7 fb = f(b);
8
9 if fa * fb > 0
10     disp('Wrong guess');
11 else
12     fprintf('%-10s%-10s%-10s%-10s\n', 'a', 'b', 'c', 'f(c)');
13     fprintf('-----\n');
14
15     c = a - fa * (b - a) / (fb - fa);
16     fc = f(c);
17
18     while abs(fc) > lim
19         fprintf('%-10.4f%-10.4f%-10.4f%-10.4f\n', a, b, c, fc)
20         ;
21
22         if fa * fc < 0
23             b = c;
24             fb = f(b);
25         else
26             a = c;
27             fa = f(a);
28         end
29
30         c = a - fa * (b - a) / (fb - fa);
31         fc = f(c);
32     end

```

```

32
33 % Print final converging step
34 fprintf('%-10.4f%-10.4f%-10.4f%-10.4f\n', a, b, c, fc);
35 fprintf('Root for Equation 1 found at: %.4f\n', c);
36 end

```

Listing 1: Octave Code for $f(x) = x^3 - 2x^2 - 5$

5.2 Code for Equation 2

```

1 % False Position Method for Equation 2: f(x) = e^x - 4*sin(x) -
  3*log(x)
2 f = @(x) exp(x) - 4*sin(x) - 3*log(x);
3 a = 1; b = 2; % Initial guesses
4 lim = 0.001; % Tolerance limit
5
6 fa = f(a);
7 fb = f(b);
8
9 if fa * fb > 0
10     disp('Wrong guess');
11 else
12     fprintf('%-10s%-10s%-10s%-10s\n', 'a', 'b', 'c', 'f(c)');
13     fprintf('-----\n');
14
15     c = a - fa * (b - a) / (fb - fa);
16     fc = f(c);
17
18     while abs(fc) > lim
19         fprintf('%-10.4f%-10.4f%-10.4f%-10.4f\n', a, b, c, fc);
20         ;
21
22         if fa * fc < 0
23             b = c;
24             fb = f(b);
25         else
26             a = c;
27             fa = f(a);
28         end
29
30         c = a - fa * (b - a) / (fb - fa);
31         fc = f(c);
32     end
33
34 % Print final converging step
35 fprintf('%-10.4f%-10.4f%-10.4f%-10.4f\n', a, b, c, fc);
36 fprintf('Root for Equation 2 found at: %.4f\n', c);
37 end

```

Listing 2: Octave Code for $f(x) = e^x - 4\sin(x) - 3\ln(x)$

6 Output and Graphs

6.1 Program Outputs (Command Window)

The outputs obtained from the Octave terminal, showing the iteration tables and final roots for both equations, are attached below.

```
octave:7> lab_2_eq_1

a          b          c          f(c)
-----
2.0000    3.0000    2.5556   -1.3717
2.5556    3.0000    2.6691   -0.2338
2.6691    3.0000    2.6873   -0.0363
2.6873    3.0000    2.6901   -0.0056
2.6901    3.0000    2.6906   -0.0008
Root For Equation 1 Found at: 2.6906
octave:8> |
```

Figure 1: Octave Terminal Output and Iteration Table for Equation 1 (False Position)

```
lab_2_eq_2

a          b          c          f(c)
-----
1.0000    2.0000    1.2791   -0.9761
1.2791    2.0000    1.5448   -0.6163
1.5448    2.0000    1.6674   -0.2169
1.6674    2.0000    1.7056   -0.0610
1.7056    2.0000    1.7159   -0.0160
1.7159    2.0000    1.7186   -0.0042
1.7186    2.0000    1.7193   -0.0011
1.7193    2.0000    1.7195   -0.0003
Root For Equation 2 Found at: 1.7195
octave:9> |
```

Figure 2: Octave Terminal Output and Iteration Table for Equation 2 (False Position)

6.2 Graphical Representation

The following graphs illustrate the functions crossing the x-axis, visually confirming the roots obtained computationally.

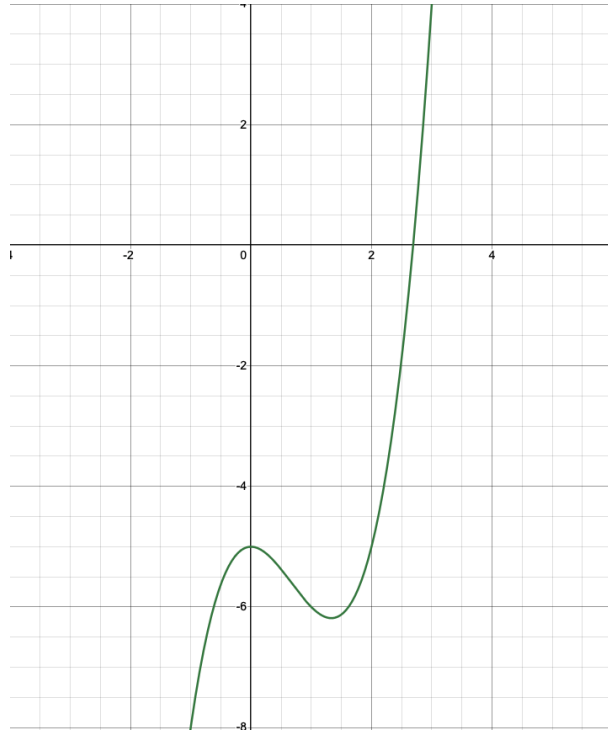


Figure 3: Plot of Equation 1: $f(x) = x^3 - 2x^2 - 5$

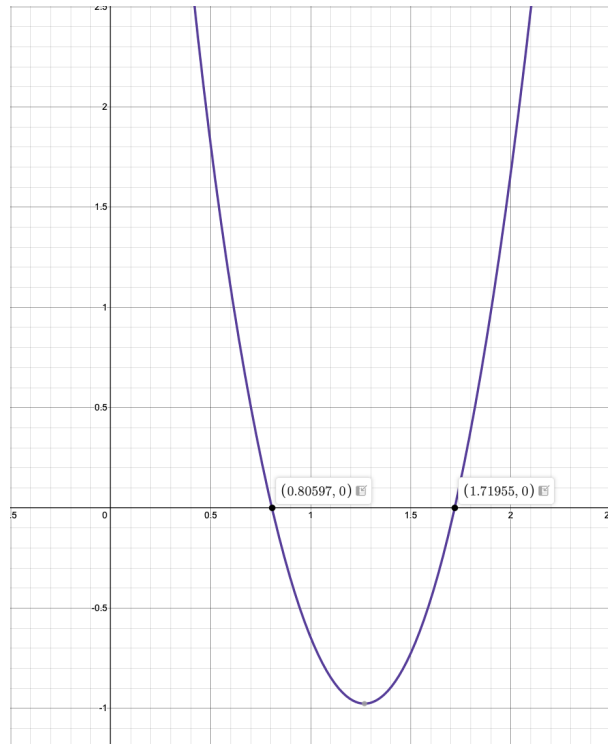


Figure 4: Plot of Equation 2: $f(x) = e^x - 4 \sin(x) - 3 \ln(x)$

7 Discussion

In this experiment, the False Position (Regula Falsi) method was implemented to find the roots of two non-linear equations. Unlike the Bisection method which strictly halves the interval, the False Position method uses the function values at the endpoints to draw a secant line, leading to a more targeted approximation of the root.

For the polynomial equation ($x^3 - 2x^2 - 5 = 0$), the initial guesses $a = 2$ and $b = 3$ were used. For the transcendental equation ($e^x - 4\sin(x) - 3\ln(x) = 0$), the initial guesses were $a = 1$ and $b = 2$. In both instances, the method successfully converged to the true root within the specified tolerance limit of $lim = 0.001$. As expected, the False Position method converged relatively efficiently by utilizing the magnitudes of $f(a)$ and $f(b)$ rather than just their signs.

8 Conclusion

The implementation of the False Position method successfully demonstrated how secant-based interpolation improves upon the pure bracketing approach of the Bisection method. By generating numerical solutions in Octave and corroborating them with graphical plots, the reliability and mechanics of this root-finding algorithm were clearly verified.

References

- [1] Department of Electrical & Computer Engineering, RUET. *Lab Manual: Discrete Mathematics & Numerical Methods (ECE 2214)*. Rajshahi University of Engineering & Technology.